Ejercicio 1.

```
xTaskCreate(tareaRed, "RED", 3072, NULL, 3, &hRed);
xTaskCreate(tareaLog, "LOG", 2048, NULL, 1, &hLog);
```

Pregunta. Indique qué representan los seis argumentos, qué tarea tiene mayor prioridad y si la creación garantiza que RED se ejecutará permanentemente.

Respuesta.

a) Argumentos: (1) función de la tarea; (2) nombre; (3) tamaño de pila (en bytes en ESP-IDF/Arduino-ESP32); (4) parámetro; (5) prioridad; (6) dirección donde se guarda el handle.

b) RED (prioridad 3) tiene mayor prioridad que LOG (1).

c) No. RED puede bloquearse, suspenderse o ser desplazada por una tarea de mayor prioridad. Además, antes de asumir que se creó hay que verificar que retorne `pdPASS`.

Ejercicio 2.

```
void A(void *p) { for(;;) { procesar(); vTaskDelay(pdMS_TO_TICKS(10)); } }
void B(void *p) { for(;;) { registrar(); vTaskDelay(pdMS_TO_TICKS(50)); } }
```

Pregunta. A tiene prioridad 4 y B prioridad 2. A acaba de llamar a vTaskDelay(10 ms) y B está lista. ¿Quién ejecuta y qué ocurre cuando vence el retraso de A?

Respuesta.

a) Ejecuta B: A pasa de Running a Blocked y B pasa de Ready a Running.

b) A los 10 ms, A vuelve a Ready y, por tener mayor prioridad, desplaza inmediatamente a B. `vTaskDelay` no consume CPU, solo bloquea la tarea.

Ejercicio 3.

```
for (;;) { trabajo(); vTaskDelay(pdMS_TO_TICKS(20)); }
```

Pregunta. El trabajo dura 7 ms constantes. Calcule el intervalo entre inicios de trabajo. ¿Cómo mantener un período de 20 ms?

Respuesta.

a) Intervalo = 7 + 20 = 27 ms, porque `vTaskDelay` espera 20 ms *después* de terminar el trabajo (genera deriva).

b) Usar `vTaskDelayUntil`, que toma como referencia el inicio anterior:

```
TickType_t ant = xTaskGetTickCount();
for(;;){ trabajo(); vTaskDelayUntil(&ant, pdMS_TO_TICKS(20)); }
```

El trabajo debe terminar antes del siguiente instante previsto.

Ejercicio 4.

```
t=0: A(prio 3) ejecuta 4 ms y bloquea 8 ms.
B(prio 2) requiere 6 ms de CPU y luego termina.
C(prio 1) requiere 3 ms de CPU y luego termina.
```

Pregunta. Construya el intervalo de ejecución entre t=0 y t=13 ms. Considere que el bloqueo de A comienza en t=4 y vence en t=12; al despertar A necesita 2 ms más de CPU.

Respuesta.

$[0, 4)$**:** A (mayor prioridad). $[4, 10)$**:** B, termina. $[10, 12)$**:** C, ejecuta 2 de sus 3 ms.

$[12, 13)$**:** A despierta, vuelve a Ready y desplaza a C; ejecuta 1 de sus 2 ms (terminaría en t=14).

C queda en Ready con 1 ms pendiente. La prioridad solo decide entre tareas listas.

Ejercicio 5.

```
void alta(void *p) { while (true) { calcular(); } }
void baja(void *p) { while (true) { atender(); } }
```

Pregunta. alta tiene prioridad 5, baja prioridad 1, ambas fijadas al mismo núcleo. Explique qué se observa y proponga una corrección.

Respuesta.

a) *alta* nunca se bloquea y siempre está Ready/Running, así que *baja* sufre **inanición**: nunca obtiene CPU.

b) Corrección en *alta* (no en *baja*): que espere un evento, cola o notificación, o use `vTaskDelay` si es periódica; o replantear las prioridades. `yield()` no garantiza que corra una tarea de menor prioridad.

Ejercicio 6.

```
A y B: prioridad 2; ambas listas, sin bloqueos.
C: prioridad 3, bloqueada hasta t=5 ms.
```

Pregunta. Explique cómo se distribuye CPU antes y después de t=5 ms. ¿Puede predecirse el orden exacto de las impresiones Serial?

Respuesta.

a) Antes de t=5: A y B comparten el núcleo por *time slicing* (turnos según el tick).

b) En t=5: C se desbloquea y desplaza a A y B por tener mayor prioridad.

c) No. El puerto serie añade latencia, bloqueos e intercalados, así que el orden exacto no es predecible.

Ejercicio 7.

```
xTaskCreate(tarea, "T", 2048, NULL, 2, &h);
vTaskSuspend(h);
/* transcurren 100 ms */
vTaskResume(h);
```

Pregunta. Describa las transiciones de estado posibles y explique si el paso de 100 ms despierta por sí solo a T.

Respuesta.

a) Creación → Ready (puede pasar a Running) → `vTaskSuspend` → Suspended → `vTaskResume` → Ready; correrá cuando el planificador la elija.

b) No. Una tarea Suspended no despierta con el tiempo; solo con `vTaskResume`. Eso la diferencia de Blocked, que sí sale al vencer un tiempo o llegar un evento.

Ejercicio 8.

```
Tarea X: prioridad 4, espera 100 ms por ciclo.
Tarea Y: prioridad 1, calcula durante 15 ms y espera 5 ms.
```

Pregunta. ¿Es correcto afirmar que X consume más CPU por tener prioridad mayor? Fundamente.

Respuesta. No. La prioridad define quién ejecuta primero cuando ambas están listas, no cuánta CPU recibe. X pasa casi todo el ciclo bloqueada y consume poco; Y aprovecha la CPU mientras X está bloqueada. La carga real solo se conoce midiendo.

Ejercicio 9.

```
BaseType_t r = xTaskCreate(tarea, "T", 1024, NULL, 2, NULL);
if (r != pdPASS) Serial.println("ERROR");
```

Pregunta. ¿Qué significa ERROR? ¿Es correcto concluir que siempre faltó memoria? ¿Qué otro riesgo presenta una pila pequeña?

Respuesta.

a) Que `xTaskCreate` no pudo crear la tarea.

b) No siempre. La falta de memoria es la causa habitual, pero hay que diagnosticarlo antes de afirmarlo.

c) Desbordamiento de pila durante la ejecución, aunque la creación haya sido exitosa. Ojo: en ESP-IDF la pila va en bytes; en FreeRTOS estándar, en palabras.

Ejercicio 10.

```
#define CONFIG_TICK_RATE_HZ 100
vTaskDelay(pdMS_TO_TICKS(15));
```

Pregunta. Con tick de 10 ms, ¿cuántos ticks produce típicamente pdMS_TO_TICKS(15)? ¿Se garantiza una espera exacta de 15 ms?

Respuesta.

a) $15 \cdot 100/1000 = 1.5 \rightarrow$ **1 tick** (división entera).

b) No. 1 tick = 10 ms y la espera real depende de en qué punto del tick se llama (fase). Para garantizar un mínimo hay que considerar la resolución y el redondeo.

Ejercicio 11.

```
void monitor(void *p) { for (;;) { Serial.println("OK"); vTaskDelay(pdMS_TO_TICKS(100)); } }
```

Pregunta. El monitor serie registra intervalos de 101, 107, 99 y 112 ms. Dé tres causas plausibles y diseñe una medición mejor.

Respuesta.

a) Causas: (1) tiempo de `Serial.println` y su buffer; (2) interferencia de tareas de mayor prioridad; (3) variación del planificador/tick y carga del sistema.

b) Medición: tomar marcas con `esp_timer_get_time()` y calcular diferencias, separar la medición del envío por serie, repetir muchas muestras y reportar mínimo, máximo y promedio. Estos datos no prueban que FreeRTOS falló.

Ejercicio 12.

```
Tarea RX (prioridad 4): recibe tramas cada 10 ms.
Tarea PROCESA (prioridad 3): tarda 6 ms por trama.
Tarea LOG (prioridad 1): registra resultados; tarda 3 ms por registro.
```

Pregunta. Proponga estados y prioridades razonables. ¿Qué pasaría si RX sondea permanentemente sin bloquearse? ¿Qué mediciones usaría para validar el diseño?

Respuesta.

a) Prioridades RX > PROCESA > LOG son razonables (RX es la más sensible al tiempo). Cada tarea debe quedar **Blocked** mientras espera: RX por recepción/evento, PROCESA por mensajes y LOG por registros.

b) RX con prioridad 4 acapara la CPU; PROCESA y LOG no ejecutan, se acumulan datos y se pierden tramas. La prioridad no reemplaza un diseño con bloqueo.

c) Medir: tasa de recepción, latencia RX→procesamiento, ocupación y desbordamiento de buffers, uso de CPU y variación temporal bajo carga.